EASILE 2D Anti-Cheat Game

PATCH-01 공격·방어 통합 실험 보고서

Ultimate Cooldown 10초 → 1초 Binary Patch

Baseline	EASILE Baseline 1.0
실험 번호	PATCH-01
실험일	9.30 ~ 10.
작성자	정명훈
공격 대상	Ultimate Cooldown
공격 목표	10초 → 0초
방어 단계	Anti-Patch Defense
최종 판정	☐ 공격 성공 ☐ 방어 성공 ☐ 부분 성공 ☐ 실패

작성 가이드: 이 문서는 한 번의 보안 반복 실험 전체를 기록한다. 패치 공격 성공만 기록하지 말고, 방어 코드 구현과 동일 공격 재시도까지 완료한 뒤 최종 결론을 작성한다.

1. 실험 개요

프로젝트명	EASILE 2D Anti-Cheat Game
Baseline 상태	Baseline 1.0 – CONFIRMED
실험 목적	궁극기 쿨타임을 10초에서 1초로 변경하는 패치 공격과 그에 대한 방어를 반복 검증한다.
공격 유형	Binary Patch / Value 또는 Logic Patch
대상 빌드	☐ Debug ☐ Release ☐ Both
분석 도구	ida, xdbg64
패치 도구	xdbg64
개발/방어 도구	Visual Studio 2022 / 기타: _____

1.1 실험 목표

- Baseline 1.0 실행 파일에서 Ultimate Cooldown 관련 위치를 식별한다.
- 정상 10초 쿨타임을 0초로 변경하여 실제 게임 동작을 변조한다.
- 패치 전후 결과를 동일 조건에서 비교하여 공격 성공을 검증한다.
- 패치가 가능했던 원인을 분석하고 적절한 방어 전략을 선정한다.
- 방어 코드를 구현하고 정상 동작 및 오탐 여부를 확인한다.
- 동일 패치 공격을 다시 수행해 탐지 또는 차단 여부를 검증한다.

1.2 Baseline 확인

☐ 사용자 직접 플레이 검증 완료

☐ Debug 빌드 정상

☐ Release 빌드 정상

☐ 기존 자동 테스트 PASS

☐ Baseline 원본 EXE 별도 보존

☐ Baseline EXE SHA-256 기록

Baseline 관련 비고

2. 실험 환경 및 재현 정보

OS	Windows 64-bit
CPU / 환경	본인 PC 사양
프로젝트 Commit / Tag	Baseline 1.0

Debug EXE 경로	배포된 exe파일을 참조하면 됨
Release EXE 경로	배포된 exe파일을 참조하면 됨
Baseline SHA-256	4AC67F48BE58140159375027C69BCF65A22696E1C62CDD04CDEE38BD5AEC5DC9
Patched SHA-256	18CA7DEFC6324C7409A62F248B15A109C2D8FC4ED9A05E7C5282518E13D35F73
Defense Build SHA-256	

작성 가이드: 가능하면 공격 전·후·방어 적용 후의 실행 파일을 각각 보존하고 SHA-256을 기록한다. 이렇게 해야 실험 결과를 나중에 재현할 수 있다.

3. 공격 전 정상 동작 측정

3.1 정상 조건

정상 Ultimate Cooldown: 10 seconds

측정 항목	1회	2회	3회	평균
스킬 쿨타임	10초	10초	10초	10초

측정 방법 / 입력 조건 / 위치 / 상대 상태

게임에서 스킬을 누른 뒤에 좌측 상단에 나오는 대기 시간을 통해 측정함/스킬 키 누름/위치는 상관 없음/상대 상태 역시 상관이 없다

Baseline 관찰 결과

늘 스킬 키를 누른 다음에 10초 뒤에 다시 스킬을 재사용 가능하게 된다.

4. 패치 공격 분석 과정

4.1 소스 기준 후보 위치

관련 클래스/함수	Hero::UltimateReady(), Hero::StartUltimate(), GameWorld의 Ultimate 발사 조건
관련 변수	ultimateCd_
정상 값	10초
추가 확인 사항	컴파일 최적화로 인해 Release 바이너리에서는 소스 구조와 동일하게 보이지 않을 수 있음

4.2 바이너리 탐색 과정

작성 가이드: 단순히 정답 주소만 적지 말고, 어떤 단서로 후보를 좁혔는지 순서대로 기록한다. 예: 문자열/상수/함수 흐름/XREF/동적 관찰.

단계	수행 내용	관찰 결과	다음 판단
1단계	함수 이름 중 공격과 관련된 함수 이름을 찾으려 function name중 attack이 들어간 함수를 찾아봄	실제로 그 중에서 Input Manager::ReadAttackEdges.함수에서ultimate라는 시간과 관련된 값을 찾았다.	하지만 함수의 내부 로직을 분석해 본 결과 이건 궁극기 부분에 대해 keyJ && !previousUltimate_만 검사해서 J가 이번 프레임에 새로 눌렸는지만 판단하고 코드 안에 10.0, 10000, deltaTime, 현재 시간 비교 같은 시간 계산이 전혀 없는 걸 봐서 공격 주기와 관련된 함수가 아니라는 걸 알 수 있었다.
2단계	ida의 subview중 local types에 들어가서 Hero구조체를 들여다 봄	Hero구조체에 스킬의 쿨타임 정보와 관련되어 보이는ultimateCd_변수를 발견함(위치 0x40도 파악함)	이 ultimatecd_변수를 alt + t로 검색해보기
3단계	이 ultimatecd_변수를 alt + t로 검색해봤다.	아무것도 찾지 못했다.	Hero관련된 함수에서 한 번 찾아보기로 결정을

단계	수행 내용	관찰 결과	다음 판단
			했다.
4 단계	Hero관련된 함수에서 한 번 찾아보기로 결정을 했다.	update라는 함수에서 뭔가 계속 상태를 업데이트하는 것을 확인했다.	update함수에서 HeroultimateCd_변수를 찾아보기로 결정함
5단계	update함수에서 HeroultimateCd_변수를 찾아봤다.	찾아본 결과 실제로 Hero.ultimateCd_변수를 찾았다.	이제 그 부분에서 다시 쿨타임 시간을 설정하는 부분을 10초에서 0초로 패치하기로 결정을 내렸다.

4.3 최종 패치 지점

Module	GameClient.exe
Function / 추정 함수	GameWorld::Update(PlayerCommand const&, float)
RVA	0x1553E
VA	0x140015534
관련 Instruction	mov rax, 4024000000000000h
원본 Bytes	48 B8 00 00 00 00 00 00 24 40
변경 Bytes	48 B8 00 00 00 00 00 00 00 00

관련 디스어셈블리 / 의사코드

```
if ( resolveAim[1] && *((double *)v4._Ptr + 8) <= 0.000001 )
    {
      v95 = *((_DWORD *)v4._Ptr + 14);
      if ( v4._Ptr[1056] )
      {
        v96 = GameWorld::AutoAim((GameWorld *)v4._Ptr, v112, 1000.0, nullptr);
      }

 *((_QWORD *)v4._Ptr + 8) = 0x4024000000000000LL;
```

5. 패치 수행 내용

항목	Before	After

항목	Before	After
쿨타임 시간 10초 -> 0초로 변경	*((_QWORD *)v4._Ptr + 8) = 0x4024000000000000LL;	*((_QWORD *)v4._Ptr + 8) = 0x0LL;

실제 패치 절차

1.우선 x64dbg에서 해당 부분을 찾기 위해서 ida와 주소를 일치시킨다음에 ida에서 해당 부분의 주소 정보를 얻어서 x64dbg에서 그 위치로 갔다.
2. 그 다음에 그 부분을 위에서 언급한 바이트로 교체함 -> 잔존 바이트는 nop operation으로 매꿈
3.이렇게 패치를 한 다음에 파일의 이름을 gameclient_patched.exe로 바꿈
4.이렇게 저장된 파일에서 다시 패치된 바이트를 재확인함
5.파일을 실제로 실행시켜 쿨타임이 변경되었는지를 확인함
6.그 다음에 sha-256을 재계산해서 입력함

☐ 원본 EXE 백업 완료
☐ 패치 후 별도 파일명으로 저장
☐ 패치된 바이트 재확인
☐ 파일 실행 가능 여부 확인
☐ SHA-256 재계산

6. 패치 공격 결과 검증

측정 항목	Baseline	Patched	판정
스킬 쿨타임	10초	0초	패치 내용이 잘 반영됨
게임 정상 실행 여부	정상 실행됨	정상 실행됨	정상
Ultimate 공격 기능	데미지와 기능이 그대로 유지됨	baseline과 동일함	정상

6.1 성공 판정 기준

☐ Baseline에서는 약 10초마다 Ultimate 사용 가능
☐ Patched에서는 약 0초마다 Ultimate 사용 가능
☐ 게임이 정상 실행됨
☐ Ultimate 공격 기능 자체가 정상 유지됨
☐ 변경된 동작이 패치한 쿨타임 로직 때문임을 확인

공격 최종 판정	SUCCESS

패치 후 플레이 영향 / 예상치 못한 부작용

게임을 굉장히 빠르게 끝낼 수 있게 되었다. / 부작용은 딱히 없다 애초에 쿨타임만 변경해서 나머진 동일하다.

7. 공격 원인 및 취약 지점 분석

발견 난이도	보통
패치 난이도	매우 쉬움
검증 난이도	쉬움

왜 이 패치가 가능했는가?

이 패치가 가능했던 이유는 이 파일에 이름을 숨기는 난독화나 optimize가 심하게 들어가지 않았고 sha검증 코드와 같은 무결성 검증 안티디버깅 기법이 들어가지 않았기 때문에 발견에는 생각보다 시간이 많이 들어갔지만 실제 패치와 검증은 쉽게 할 수 있었다.

공격자가 알아야 했던 정보 / 가장 중요한 단서

가장 중요한 단서는 local type중에서 Hero class에서 이런 쿨타임 관련된 변수를 가지고 있다는 점과 이 점을 통해서 hero관련 메소드에서 이런 쿨타임 정보를 이용해 스킬 발사 여부를 결정하는 함수 update함수를 찾는 것이었다.

현재 구조에서 공격자가 우회할 수 있는 다른 경로

현재 구조는 애초에 굉장히 보안에 취약한 구조라서 그냥 local types에서 그 변수를 찾고 그 변수를 조작하는 데에 가서 값을 패치하면 쉽게 우회할 수 있다. 그걸로 체력,경험치, 공격력 등 다양한 변수들을 내가 원하는 값으로 변경할 수 있다.

8. 방어 전략 설계

8.1 후보 방어 기법 비교

방어 후보	원리	장점	한계	선택 여부
EXE/코드 무결성 검사	실행 중 특정 코드 영역이나 함수 바이트의 hash/CRC를	지금 수행한 바이너리 패치 자체를 직접 탐지가능	검사 코드까지 패치하면 우회 가능	o

방어 후보	원리	장점	한계	선택 여부
	계산해서 원본과 비교			
Critical Value Validation	ultimateCd_가 정상 범위인지 검사. 예: 10초 설정 직후 값이 비정상적으로 작으면 복구/차단	구현 매우 쉬움, 현재 공격에 정확히 대응	검사 코드나 기준값도 함께 패치 가능	x
Runtime Cooldown Validation	ultimateCd_변수만 믿지 않고, 마지막 궁극기 사용 시각과 현재 시각의 차이가 10초 이상인지 별도로 확인	0초/1초/5초 등 값 패치를 모두 막을 수 있음	시간 검증 로직까지 패치하면 우회 가능	o
중복 상태 검증	ultimateCd_+ lastUltimateTime같이 서로 다른 상태 2개를 동시에 검사	한 변수만 패치하는 공격에 강함	두 로직 모두 분석되면 패치 가능	x
상수/로직 분산	10.0을 한 곳에 직접 넣지 않고 여러 값이나 계산식으로 생성	IDA에서 10.0 상수 찾기가 어려워짐	본질적인 방어라기보다 난독화에 가까움	x

작성 가이드: 예: 코드/함수 무결성 검사, Critical Value Validation, 실제 발사 시각 기반 Runtime Cooldown Validation, 중복 상태 검증 등을 비교한다.

8.2 최종 선택 방어

선택한 방어 기법	EXE/코드 무결성 검사 + Runtime Cooldown Validation
보호 대상	Ultimate Cooldown
탐지 대상	궁극기를 10초 안에 사용하는 것 + 파일의 바이트값이 보존되었는지 확인
탐지 시 행동	기능 차단 및 종료, 궁극기 쿨타임 값 복구
선택 이유	하나는 정적인 방어 방법이고 하나는 동적으로 방어하기 때문에 시너지가 좋을 거 같아서 위 두 개를 선택했다.

방어 로직 설계 설명

우선은 GameWorld::Update함수에 대해서만 코드 무결성 검사를 하기로 마음을 먹고 이를 위해서 가장 먼저 gameworld.cpp에 검증 함수가 담긴 해더를 넣고 별도 cpp와 header파일 내에 원본 파일의(release기준)의 update함수의 rva, 함수의 바이트 수를 ida에서 alt + p로 알아내 그 길이만큼 crc32로 각 바이트마다 xor +

비트마다 추가 연산을 해서 단일 crc32값을 만들고 게임 실행시 스킬을 눌렀을 때 crc32를 축정해서 둘이 같지 않으면 버그 유저에게 패널티로 아예 스킬을 사용하지 못하게 막았다. 이는 정적인 보호방법이다, 그 다음에는 Runtime cooldown validation을 위해서 crc32검증을 통과한 이후에 런타임에서 단순히 현재 쿨타임 시간 변수를 보고 궁극기 발사 여부를 판정하는 것이 아니라 마지막 궁극기를 사용한 시간에서 현재 시간을 뺀 값을 이용해서 10초가 지나지 않았으면 궁극기가 발사가 되지 않도록 코드를 수정할 계획이다.

정상 상태와 변조 상태를 구분하는 기준

update함수의 바이트 값의 변조 여부로 구분이 가능하고 마지막으로 궁극기를 사용한 시간과 현재 시간의 차이로 정상 상태와 변조 상태를 구분하려고 한다.

예상 가능한 우회 방법과 한계

예상 가능한 우회 방법은 crc32는 현재 crc32와 예상 crc32가 같은지 비교하는 부분을 무조건 true를 반환하게 하고 또한 runtime도 조건을 not을 넣거나 무조건 true가 되게 하거나 기준 값을 10초에서 0초로 바꾸는 등의 방식으로 우회가 가능하다는 한계가 존재한다. 하지만 이전의 코드보다는 시간이 더 소요될 것이다.

9. 방어 코드 구현 과정

9.1 수정 파일 및 함수

파일	클래스/함수	변경 내용	목적
src/IntegrityGuard.h	IntegrityGuard	CalculateCRC32(), VerifyUpdateIntegrity() 등 무결성 검사 함수 선언 추가	실행 코드 변조 여부를 검사하기 위한 무결성 검사 인터페이스 정의
src/IntegrityGuard.cpp	CalculateCRC32(), VerifyUpdateIntegrity()	GameWorld::Update()코드 영역에 대해 CRC32를 계산하고 정상 Release 빌드의 기준 CRC와 비교하도록 구현 (UPDATE_RVA=0x14480 ,UPDATE_SIZE=0x13B4)	EXE 패치를 통한 GameWorld::Update()변조 탐지
src/GameWorld.h	GameWorld	lastUltimateUseTime_멤버 변수 추가 및 초기값 -10.0설정	기존 ultimateCd_와 독립적인 궁극기 사용 시각을 저장하여 우회 패치에 대비
src/GameWorld.cpp	GameWorld::Initialize()	simulationTime_및 lastUltimateUseTime_초기화 추가	게임 시작/재시작 시 런타임 쿨다운 검증 상태를 정상화
src/GameWorld.cpp	GameWorld::Update()	무결성 검사 수행 후, 궁극기 입력 시 simulationTime_ - lastUltimateUseTime_ < 10.0여부를 추가 검사하고 발사 성공 시	기존 UltimateReady()가 패치되어도 별도의 10초 런타임 검증으로 연속 궁극기 사용 차단

		마지막 사용 시각 갱신	
GameClient.vcxproj/ GameTests.vcxproj	빌드 설정	IntegrityGuard.cpp/.h 를 src₩IntegrityGuard.cpp, src₩IntegrityGuard.h경로로 프로젝트에 등록하고 테스트 프로젝트에도 구현 파일이 링크되도록 수정	IntegrityGuard소스 이동 후 정상 컴파일·링크 및 테스트 빌드 지원

9.3 핵심 코드 설명

핵심 방어 코드 또는 의사코드

위의 로직 설명을 그대로 코드로 구현함 -> 생략

왜 이 위치에 방어 로직을 배치했는가?

왜냐하면 그 위치가 너무 자주 호출되지 않고 딱 j키를 눌러서 스킬이 나갈 때만 실행되게 하고 싶었기 때문이다.

10. 방어 코드 빌드 및 정상 동작 검증

검증 항목	Release	비고
방어 적용 후 정상 Ultimate 10초 쿨타임 유지	o	
일반 공격 정상	o	
이동 정상	o	

검증 항목	Release	비고
Bot 정상	o	
Tower/Minion/Nexus 진행 정상	o	
기존 테스트 PASS	o	
정상 플레이에서 오탐 없음	o	주의해야 하는 게 이게 코드를 수정하면 rva에 있는 게 update함수가 아닌 다른 함수값이 들어갈 수 있어서 그때마다. rva값을 바꿔줘야 합니다.

☐ 방어 적용 후 정상 Ultimate 10초 쿨타임 유지
☐ 일반 공격 정상
☐ 이동 정상
☐ Bot 정상
☐ Tower/Minion/Nexus 진행 정상
☐ 기존 테스트 PASS
☐ 정상 플레이에서 오탐 없음

정상 플레이 테스트 결과 / 오탐 여부

정상 플레이를 하는 경우에는 정상적으로 돌아갔습니다. 다만 파일의 내용을 바꿀 때마다 intergrity의 rva정보를 수정해야 하고 update함수를 수정시 update의 바이트 크기 정보도 수정해야 한다는 점이 좀 번거롭습니다.

11. 동일 공격 재시도

작성 가이드: 방어 적용 후 공격 조건을 가능한 한 PATCH-01 최초 공격과 동일하게 유지한다. 그래야 방어 전후를 비교할 수 있다.

재공격 대상 EXE	release의 GameClient.exe
공격 방법	Ultimate Cooldown 10초 → 0초 동일 패치

패치 위치	이전과 동일
변경 값/바이트	이전과 동일
방어 탐지 여부	탐지
공격 성공 여부	차단

항목	방어 전 공격	방어 후 재공격	차이
궁극기 쿨타임 10 -> 0초	실제로 궁극기 쿨타임 값을 10 -> 0초로 패치	동일	이번에는 crc32의 검증에 걸려 아예 궁극기를 한번도 사용을 못하게 됐다.

재공격 수행 과정

공격과정과 동일하다.

탐지/차단 로그 또는 화면 관찰 내용

j키를 눌러도 crc32검증값이 예상과 달라서 공격 자체가 나가지 않는다.

12. 방어 효과 평가

평가 항목	결과	근거
정상 게임 실행을 방해하지 않았는가?	전혀 방해하지 않는다.	실제 플레이해본 결과 문제 없었다.
기존 테스트에서 회귀가 발생하지 않았는가?	발생하지 않는다.	실제 테스트 결과 문제가 없었다.
0초 패치를 실제로 탐지 또는 차단했는가?	실제로 탐지하고 차단했다.	아예 궁극기를 한 번도 사용하지 못했다.

평가 항목	결과	근거
패치 위치가 달라져도 동일 변조를 탐지할 가능성이 있는가?	위치가 달라져도 runtime cooldown이 있어 10초 이전에 궁극기를 사용 시 여전히 탐지할 수 있다.	결과와 동일함
단순 NOP/분기 변경 등 다른 우회에 취약하지 않은가?	솔직히 좀 취약하긴 하다.	crc32는 무조건 true반환 , 그리고 runtime cooldown은 조금 어렵긴 하겠지만 관련된 변수값을 바꾸거나 분기를 뒤집으면 생각보다 쉽게 우회가 가능하다.
성능 저하가 허용 가능한 수준인가?	허용 가능한 수준이다.	궁극기가 사용될 때에만 이 방어로직들이 동작하기 때문이다.

12.1 평가 질문

☐ 정상 게임 실행을 방해하지 않았는가?
☐ 기존 테스트에서 회귀가 발생하지 않았는가?
☐ 0초 패치를 실제로 탐지 또는 차단했는가?
☐ 패치 위치가 달라져도 동일 변조를 탐지할 가능성이 있는가?
☐ 단순 NOP/분기 변경 등 다른 우회에 취약하지 않은가?
☐ 성능 저하가 허용 가능한 수준인가?

방어의 장점

정적 + 동적 방어가 들어가서 시너지가 날 수 있고 조금 더 이러한 방어방법을 우회하는데 시간이 더 소요됨

방어의 한계 및 우회 가능성

한계는 여전히 프로그래머가 시간만 어느정도 쓰면 손쉽게 무력화가 가능하다는 거다.

추가 개선 아이디어

이를 막기 위해서 crc32 -> sha256검증으로 바꾸고 그 외에도 여러 변수의 값을 여러군데 backup하고 난독화하며 안티디버깅 기법도 여러군데 넣을 필요가 있다.(하지만 너무 리버싱 난이도가 높아진다는 한계가 있다.)

13. 최종 결론

공격 결과	성공
방어 결과	성공
오탐	없음
정상 기능 회귀	없음
다음 단계	dll injection을 통한 공격과 이에 따른 방어

13.1 공격 실험 요약

Release 실행 파일을 정적 분석하여 궁극기 쿨타임을 설정하는 코드를 식별하고, 기존 10초(10.0)값을 0초(0.0)로 변경하는 바이너리 패치를 수행하였다. 패치 후 게임을 실행하여 궁극기를 쿨타임 없이 연속 사용할 수 있음을 확인하였으며, 이를 통해 단순 클라이언트 측 쿨다운 로직이 실행 파일 변조에 의해 우회될 수 있음을 검증하였다.

13.2 방어 구현 요약

패치 공격에 대응하기 위해 코드 무결성 검사와 독립적인 런타임 쿨다운 검증을 함께 적용하였다. IntegrityGuard에서 GameWorld::Update()코드 영역의 CRC32를 검사하여 실행 파일 변조를 탐지하도록 하였으며, 별도로 lastUltimateUseTime_을 이용해 마지막 궁극기 사용 시점부터 10초가 지나지 않으면 발사를 차단하도록 구현하였다. 따라서 기존 쿨다운 값이 패치되더라도 추가 검증을 통해 연속 사용을 방지할 수 있도록 방어 구조를 강화하였다

13.3 공격 → 방어 → 재공격에서 얻은 결론

스터디에서는 공격 기법 위주로 배웠는데 이번 기회에 다양한 방어 기법에 대해서 알게 되었고 이를 실제 내가 만든 게임에 적용하면서 생각보다 이러한 기법들을 적절히 적용하는 게 난이도가 높고 그럼에도 불구하고 우회가 생각보다 쉽게 되서 허무하게 뚫릴 수 있다는 걸 알게 되었다. 앞으로는 방어 기법에 대해서도 더 알아보고 적용하는 연습을 더 해야겠다고 느꼈다.

13.4 다음 실험에서 개선할 점

15. 다음 보안 반복 실험 계획

우선순위	다음 실험	목적	선행 조건

작성 가이드: 이번 PATCH-01을 완료한 뒤에는 다른 Value/Logic Patch 또는 DLL Injection 단계로 이동하되, 이번 방어의 우회 가능성을 한 번 더 검토한 후 진행한다.